

Project Execution Manual

This Project Execution Manual explains the step-by-step execution process of the project “FactCheck – DeepFake Video Detection Using ResNet-50”. It describes the complete workflow starting from environment setup, dataset preparation, model execution, API deployment, and result generation. This manual focuses on the execution phase, commands used, tools, frameworks, and runtime flow.

System Environment Setup

1. Operating System: Windows 11
2. Programming Language: Python 3.13.6
3. Development Tools: Visual Studio Code (VS Code)
4. Frameworks and Libraries
 - Backend Framework: Flask
 - Deep Learning Framework: TensorFlow
 - Machine Learning Libraries: Scikit-learn, XGBoost
 - Computer Vision Library: OpenCV
 - Data Handling: NumPy
 - Model Persistence: Joblib
 - Database: PostgreSQL
 - Frontend Runtime: Node.js and npm

Dataset Preparation

Datasets used for the project are:

1. FaceForensics++ Dataset (C23 compression level)
2. DeepFake Detection Dataset (DFD – Google)

Dataset Organization

- Videos are categorized into two folders: real/ and fake/

Execution Workflow

The execution of the project follows a structured pipeline consisting of multiple stages. The execution of the FactCheck DeepFake Video Detection system is carried out in a systematic and sequential manner to ensure reliable processing and accurate prediction results.

1. The first step involves setting up the development environment. Python 3.13.6 is installed on a Windows 11 system along with Visual Studio Code as the primary development environment. Required libraries such as TensorFlow, OpenCV, NumPy,

Scikit-learn, XGBoost, Flask, Joblib, and PostgreSQL connectors are installed using the Python package manager.

2. Once the environment is ready, the PostgreSQL database is configured to store user information and prediction results. A dedicated database is created, and connection credentials are securely defined in the Flask configuration file. This database acts as a persistent storage system that maintains prediction history, confidence scores, and video metadata for future reference.
3. Then execution begins with the user uploading a video through the web interface. The frontend sends the video file to the Flask backend using a REST API endpoint. The backend validates the file format and size before temporarily storing the video on the server.
4. The next step is frame extraction, where OpenCV is used to read the uploaded video. Instead of processing every single frame, the system extracts a fixed number of frames uniformly across the video duration. This approach reduces computational overhead while still preserving important visual variations such as facial expressions, lighting changes, and potential manipulation artifacts.
5. Each extracted frame then undergoes preprocessing to make it suitable for feature extraction. During preprocessing, frames are resized to 224×224 pixels to match the ResNet-50 input requirements. The color format is converted from BGR to RGB, and pixel values are normalized using standard preprocessing functions. This step ensures uniformity across all frames and helps improve model stability and convergence.
6. After preprocessing, feature extraction is performed using a pre-trained ResNet-50 model with its classification layers removed. The model processes each frame and produces a 2048-dimensional deep feature vector that captures high-level spatial and textural patterns. These features are effective in identifying subtle inconsistencies commonly found in manipulated videos.
7. To further strengthen detection capability, handcrafted features are extracted from each frame. These include Local Binary Patterns for texture analysis, Sobel edge features for gradient detection, FFT-based frequency features, and color intensity statistics etc. These 519 handcrafted descriptors capture fine-grained artifacts that may not be fully represented by deep learning features alone.
8. The deep features and handcrafted features are then concatenated to form a single 2567-dimensional feature vector for each frame. These feature vectors are stored efficiently using NumPy arrays, enabling faster access during training and prediction. Before

classification, the features are standardized using feature scaling techniques to ensure all features contribute equally to the learning process.

9. Machine learning classification is then carried out using Random Forest and XGBoost algorithms. Both classifiers are trained on the extracted feature vectors, allowing them to learn the distinguishing patterns between real and fake videos. Probability calibration is applied to ensure that confidence scores accurately reflect prediction reliability.
10. Once the models are trained, they are serialized and saved using Joblib for reuse during inference. During prediction, the stored models analyze the extracted features from the uploaded video. Frame-level predictions are aggregated to produce a final video-level decision. If the mean fake probability exceeds the defined threshold, the video is classified as a deepfake; otherwise, it is marked as authentic.

The final step involves presenting the results to the user through the web interface. The prediction label and confidence score are displayed clearly, and the results are stored in the PostgreSQL database. This completes the execution cycle, ensuring traceability, accuracy, and smooth interaction between all system components.

Frontend Execution

- Frontend execution of the project requires installing frontend dependencies:
npm install
- To run the frontend application, a command needs to be run in the terminal:
npm start

Result Storage

After the prediction process is completed, the generated results are securely stored in the PostgreSQL database. The stored information includes the video identifier, prediction label (real or fake), confidence score, and timestamp. This persistent storage allows users to view past predictions through the application interface and supports future analysis and system evaluation.

Execution Outcome

Upon successful execution, the system classifies the uploaded video as either real or deepfake based on the aggregated frame-level predictions. Along with the classification result, a confidence score is generated to indicate the reliability of the prediction. The final output is displayed to the user through the web interface, completing the execution cycle of the project.

This Project Execution Manual outlines the complete execution lifecycle of the deepfake detection system. By following the above steps, the system ensures smooth data processing, reliable feature extraction, accurate classification, and user-friendly interaction through a web-based interface.